

Taller 4 - Inteligencia Artificial

Andrés Felipe Aguirre Garzon-202514009

Laura Daniela Acosta Suarez- 202514694

Santiago Garzon García- 202512373

Samuel Fernando Penha- 202413213

Punto 2. Análisis comparativo (estados explorados, longitud) en ≥ 2 layouts

```
FS -l tinyBase -q
=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]
=====
Planificador: forwardBFS

Tiempo de planificación: 0.018s
Estados expandidos: 345
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...
```

```
=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: openRescue (12x10)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(4, 2)]
=====
Planificador: forwardBFS

Tiempo de planificación: 12.775s
Estados expandidos: 12982
Longitud del plan: 26 acciones

Plan:
1. Move(robot, (1, 8), (1, 7))
2. Move(robot, (1, 7), (1, 6))
3. Move(robot, (1, 6), (1, 5))
4. Move(robot, (1, 5), (2, 5))
5. Move(robot, (2, 5), (3, 5))
6. Move(robot, (3, 5), (4, 5))
7. Move(robot, (4, 5), (5, 5))
8. Move(robot, (5, 5), (6, 5))
9. Move(robot, (6, 5), (7, 5))
10. Move(robot, (7, 5), (8, 5))
11. Move(robot, (8, 5), (8, 4))
```

En el layout tinyBase, el algoritmo BFS tuvo un buen desempeño porque el mapa es pequeño y el objetivo está cerca, solo necesitó explorar 345 estados y encontró un plan corto de 9 acciones en muy poco tiempo (0.018 segundos). En cambio, en el layout openRescue, el mapa es más grande por lo que el robot debe recorrer más distancia para completar la misión, por esta razón, BFS exploró 12982 estados y encontró un plan mucho más largo, de 26 acciones, tardando aproximadamente 12.775 segundos. Esto demuestra que BFS, aunque garantiza encontrar la solución más corta, tiene el problema de explorar una gran cantidad de estados cuando el espacio de búsqueda crece. Es decir, a medida que el mapa se vuelve más grande o existen más caminos posibles, la cantidad de combinaciones aumenta rápidamente y el algoritmo se vuelve menos eficiente en tiempo y memoria.

Punto 3. Comparación empírica de estados explorados vs. forwardBFS con justificación

```
=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]
=====
Planificador: backwardSearch

Tiempo de planificación: 0.038s
Estados expandidos: 0
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...
```

```
=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: openRescue (12x10)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(4, 2)]
=====
Planificador: backwardSearch

Tiempo de planificación: 60.998s
Estados expandidos: 0
Longitud del plan: 26 acciones

Plan:
1. Move(robot, (1, 8), (2, 8))
2. Move(robot, (2, 8), (3, 8))
3. Move(robot, (3, 8), (4, 8))
4. Move(robot, (4, 8), (5, 8))
5. Move(robot, (5, 8), (6, 8))
6. Move(robot, (6, 8), (7, 8))
7. Move(robot, (7, 8), (8, 8))
```

Layout	Algoritmo	Estados explorados	Longitud del plan	Tiempo
tinyBase	forwardBFS	345	9 acciones	0.018 s
tinyBase	backwardSearch	0	9 acciones	0.038 s
openRescue	forwardBFS	12982	26 acciones	12.775 s
openRescue	backwardSearch	0	26 acciones	60.998 s

En las pruebas realizadas ambos algoritmos encontraron planes válidos y con la misma longitud, lo que indica que tanto forwardBFS como backwardSearch logran obtener soluciones óptimas en estos layouts. Sin embargo, el comportamiento en términos de exploración y tiempo fue diferente. Por un lado forwardBFS expandió una gran cantidad de estados, especialmente en openRescue, donde alcanzó 12982 estados explorados debido al gran tamaño del mapa y la cantidad de posibles acciones desde el estado inicial. Mientras que backwardSearch reportó 0 estados expandidos porque trabaja regresando desde el objetivo y únicamente considera acciones relevantes para cumplirlo, evitando explorar muchos estados innecesarios.

En términos de tiempo, aunque backwardSearch exploró menos estados, el tiempo de ejecución fue mayor, especialmente en openRescue, donde tardó casi 61 segundos. Esto sugiere que el costo de realizar regresiones y manejar objetivos parciales puede ser alto, incluso si el número de estados explorados es pequeño. Por otro lado, forwardBFS fue más rápido en tiempo, pero mucho más costoso en cantidad de estados explorados. En conclusión, la búsqueda regresiva puede ser más eficiente en problemas donde el objetivo está claramente definido y pocas acciones contribuyen directamente a alcanzarlo, mientras que BFS tiende a explorar muchas posibilidades antes de encontrar la solución.

Punto 4: Comparación entre heurísticas con evidencia empírica

<pre>===== Operación Fénix - ISIS-1611 Inteligencia Artificial ===== Layout: tinyBase (5x7) Problema: SimpleRescueProblem Pacientes: ['patient_0'] Suministros: ['supplies_0'] Puestos médicos: [(1, 2)] ===== Planificador: aStarPlanner Heurística: ignorePreconditions Tiempo de planificación: 1.123s Estados expandidos: 252 Longitud del plan: 9 acciones Plan: 1. Move(robot, (1, 4), (1, 3)) 2. PickUp(robot, supplies_0, (1, 3)) 3. Move(robot, (1, 3), (1, 2)) 4. SetupSupplies(robot, supplies_0, (1, 2))</pre>	<pre>===== Operación Fénix - ISIS-1611 Inteligencia Artificial ===== Layout: tinyBase (5x7) Problema: SimpleRescueProblem Pacientes: ['patient_0'] Suministros: ['supplies_0'] Puestos médicos: [(1, 2)] ===== Planificador: aStarPlanner Heurística: ignoreDeleteLists Tiempo de planificación: 3.169s Estados expandidos: 25 Longitud del plan: 9 acciones Plan: 1. Move(robot, (1, 4), (1, 3)) 2. PickUp(robot, supplies_0, (1, 3)) 3. Move(robot, (1, 3), (1, 2)) 4. SetupSupplies(robot, supplies_0, (1, 2)) 5. Move(robot, (1, 2), (1, 1)) 6. PickUp(robot, patient_0, (1, 1)) 7. Move(robot, (1, 1), (1, 2))</pre>
--	---

Algoritmo	Heurística	Estados explorados	Longitud del plan	Tiempo
A*	ignorePreconditions	252	9 acciones	1.123 s
A*	ignoreDeleteLists	25	9 acciones	3.169 s

En ambos casos, el algoritmo A* encontró el mismo plan óptimo de 9 acciones, pero el comportamiento de las heurísticas fue diferente ya que la heurística ignorePreconditions exploró 252 estados porque realiza una estimación más simple del problema, ignorando todas las precondiciones de las acciones. Esto hace que la búsqueda tenga menos información sobre cuáles estados son realmente útiles para llegar al objetivo, por lo que termina explorando más posibilidades.

Por otro lado, la heurística ignoreDeleteLists solo exploró 25 estados, mostrando un comportamiento mucho más eficiente en términos de exploración. Esto ocurre porque esta heurística conserva más información del problema original y permite estimar mejor qué acciones acercan realmente al objetivo. Gracias a esto, A* puede enfocarse en caminos más prometedores y evitar expandir estados innecesarios. Aunque el tiempo de ejecución fue un poco mayor, la reducción en el número de estados explorados demuestra que ignoreDeleteLists es una heurística más informativa y efectiva para el dominio de rescate.

Punto 5:

En las pruebas realizadas con HTN y MultiRescueProblem, se observó que el orden en que se procesan los pacientes puede afectar la longitud total del plan. Si el robot atiende primero a un paciente que está más cerca del puesto médico o de los suministros, necesita realizar menos movimientos y el plan termina siendo más corto. En cambio, si comienza por un paciente más lejano, el robot debe recorrer más distancia y la cantidad total de acciones aumenta, lo cual demuestra que en la planificación jerárquica el orden de las tareas influye directamente en la eficiencia de la solución, especialmente cuando hay varios objetivos distribuidos en diferentes partes del mapa.

Punto 6.

Modelado y aplicabilidad

- **is_applicable(state, action):**

sirve para verificar si una acción puede ejecutarse en un estado determinado, para hacerlo, revisa que todas las precondiciones positivas de la acción estén presentes en el

estado y que ninguna precondition negativa aparezca en él. Su complejidad en tiempo es $O(p+n)$, donde p es el número de precondiciones positivas y n el de negativas, ya que debe revisar cada una de ellas. La complejidad en espacio es $O(1)$ porque no crea estructuras adicionales importantes y esta función no tiene propiedades de completitud ni optimalidad porque no es un algoritmo de búsqueda, sino una función auxiliar. Sin embargo, es muy importante dentro del sistema porque evita que el planeador ejecute acciones inválidas y ayuda a mantener consistentes los estados generados durante la planificación.

- **apply_action(state, action):**

se encarga de generar el nuevo estado después de ejecutar una acción. Primero elimina del estado actual los flujos que aparecen en la lista de borrado (`del_list`) y luego agrega los flujos de la lista de adición (`add_list`). Su complejidad en tiempo es $O(d + a)$, donde d es el número de elementos en la lista de borrado y a el número de elementos en la lista de adición, porque debe procesar ambos conjuntos. La complejidad en espacio es $O(n)$ ya que se crea un nuevo estado con los flujos resultantes. Esta función no tiene completitud ni optimalidad porque no realiza búsqueda, pero es fundamental para el planeador, ya que permite simular correctamente el cómo cambian los estados del mundo cuando el robot ejecuta acciones como moverse, recoger suministros o rescatar.

- **get_applicable_actions(state, domain, objects):**

genera todas las acciones posibles del dominio con sus parámetros ya instanciados y luego verifica cuáles pueden ejecutarse en el estado actual usando `is_applicable`. Su complejidad en tiempo es alta aprox $O(g(p+n))$, donde g es el número total de acciones instanciadas y $(p+n)$ corresponde a las precondiciones que se revisan en cada acción. Esto ocurre porque debe probar muchas combinaciones de objetos y celdas antes de encontrar las acciones válidas. Por otro lado, la complejidad en espacio es $O(g)$, ya que almacena la lista de acciones aplicables encontradas. Esta función al igual que las anteriores tampoco tiene completitud ni optimalidad porque no es un algoritmo de búsqueda, pero es una de las funciones más importantes del sistema, ya que actúa como la función de sucesores utilizada por todos los planificadores para expandir estados y explorar posibles planes.

Forward Planning

- **forwardBFS(problem):**

Esta función implementa una búsqueda en anchura sobre el espacio de estados, este algoritmo comienza desde el estado inicial y explora todos los estados alcanzables aplicando acciones válidas hasta encontrar uno que cumpla el objetivo. Su complejidad temporal en el peor caso es aproximadamente $O(b^d)$, donde b es el factor de ramificación y d la profundidad de la solución, ya que BFS puede llegar a expandir todos los estados de cada nivel antes de encontrar la meta. La complejidad espacial también es $O(b^d)$ porque debe almacenar en memoria todos los estados visitados y los nodos de la frontera. Este algoritmo es completo, ya que garantiza encontrar una solución si existe, siempre que el espacio de estados sea finito. Además, es óptimo cuando todas las acciones tienen el mismo costo, porque siempre encuentra el plan más corto en número de acciones. Sin embargo, su principal desventaja es el alto consumo de memoria y tiempo cuando el número de estados posibles crece mucho, como ocurre en problemas grandes de planificación.

Backward Planning

- **Regress(goal_set, action):**

Esta función implementa la regresión de objetivos en la planificación regresiva. Su propósito es calcular qué condiciones debían cumplirse antes de ejecutar una acción para garantizar que el objetivo actual se cumpla después de aplicarla. Primero verifica que la acción sea relevante para el objetivo y que no elimine ningún fluyente necesario, luego construye un nuevo conjunto de metas usando las precondiciones positivas de la acción. Su complejidad temporal es aprox $O(g + a + d + p)$, donde g es el tamaño del objetivo actual, a el tamaño del add list, d el del delete list y p el número de precondiciones positivas, ya que debe realizar varias operaciones de conjuntos. La complejidad espacial es $O(g + p)$, porque crea un nuevo conjunto de metas regresadas. Esta función no tiene completitud ni optimalidad por sí sola, ya que actúa como una operación auxiliar dentro de backwardSearch. Sin embargo, es importante para reducir el espacio de búsqueda, porque permite trabajar únicamente con acciones relevantes para alcanzar el objetivo en lugar de explorar todos los estados posibles.

- **BackwardSearch(problem)**

Esta función implementa búsqueda regresiva, empezando desde el objetivo y retrocediendo mediante regresiones hasta encontrar un conjunto de metas que pueda satisfacerse con el estado inicial. En lugar de explorar todos los estados posibles hacia adelante, este algoritmo solo considera acciones relevantes para alcanzar el objetivo actual lo que puede reducir considerablemente el espacio de búsqueda. Su complejidad temporal en el peor caso es aproximadamente $O(b^d)$, donde b es el número de

regresiones posibles y de la profundidad del plan, ya que sigue siendo una búsqueda exhaustiva. La complejidad espacial también es $O(b^d)$ porque debe almacenar los subobjetivos visitados y la frontera de búsqueda. El algoritmo es completo siempre que el espacio de búsqueda sea finito, porque eventualmente explora todas las regresiones posibles. También es óptimo si utiliza BFS sobre los subobjetivos y todas las acciones tienen el mismo costo, ya que encuentra el plan más corto. Una ventaja importante frente a forwardBFS es que suele expandir menos nodos en problemas donde el objetivo contiene pocos flujos específicos, porque solo trabaja con acciones relevantes para cumplir esas metas.

Heurísticas y A*

- **ignorePreconditionsHeuristic(state, goal, domain, objects):**

Estima cuántas acciones se necesitan para alcanzar el objetivo ignorando todas las precondiciones de las acciones. El algoritmo calcula primero los flujos del objetivo que aún no se cumplen y luego selecciona de manera greedy las acciones que cubren la mayor cantidad posible de esos flujos hasta satisfacerlos todos. Su complejidad temporal es aproximadamente $O(k \cdot g)$, donde k es el número de flujos insatisfechos y g el número total de acciones instanciadas, ya que en cada iteración revisa todas las acciones para encontrar la que cubre más objetivos. La complejidad espacial es $O(g + k)$, porque almacena todas las acciones generadas y los flujos pendientes. Por esto, esta heurística es admisible porque nunca sobreestima el costo real ya que al ignorar las precondiciones, el problema se vuelve más fácil que el original y el número de acciones calculado siempre será menor o igual al costo real. Pero no siempre es consistente, ya que el valor heurístico puede cambiar bruscamente entre estados consecutivos y su principal ventaja es que es rápida y sencilla de calcular, aunque suele ser poco informativa porque ignora gran parte de las restricciones reales del dominio.

- **ignoreDeleteListsHeuristic(state, goal, domain, objects):**

Estima el costo del plan resolviendo una versión relajada del problema donde las acciones nunca eliminan flujos del estado, en este modelo el estado solo puede crecer, por lo que el algoritmo usa hill-climbing para seleccionar en cada paso la acción que agrega más flujos útiles para alcanzar el objetivo. Su complejidad temporal es aproximadamente $O(k \cdot g)$, donde k es el número de iteraciones necesarias para satisfacer el objetivo y g el número de acciones aplicables revisadas en cada paso. La complejidad espacial es $O(n + g)$ porque almacena el estado relajado y las acciones aplicables. Esta heurística es admisible porque al ignorar las listas de borrado el problema se vuelve más fácil que el original, por lo que nunca sobreestima el costo real y suele ser más informativa que ignorePreconditionsHeuristic, ya que todavía respeta las precondiciones de las acciones y

representa mejor las restricciones reales del dominio. En muchos casos también es más consistente, porque el costo estimado cambia de forma más gradual entre estados consecutivos. Su principal ventaja es que guía mejor la búsqueda A*, reduciendo el número de estados explorados.

- **aStarPlanner(problem, heuristic):**

Esta función implementa el algoritmo A* para planificación hacia adelante, combinando el costo real acumulado del camino ($g(n)$) con una estimación heurística ($h(n)$) para decidir qué estados explorar primero, usando la función ($f(n)=g(n)+h(n)$). Su complejidad temporal en el peor caso sigue siendo aproximadamente ($O(b^d)$), donde b es el factor de ramificación y d la profundidad de la solución, aunque en las pruebas realizadas suele explorar muchos menos estados que BFS gracias a la heurística. La complejidad espacial también es ($O(b^d)$), ya que debe almacenar la frontera y los estados visitados junto con sus costos. El algoritmo es completo siempre que los costos de las acciones sean positivos, porque eventualmente explora todos los estados necesarios. Y es óptimo si la heurística utilizada es admisible y consistente, ya que garantiza encontrar el plan de menor costo. La principal ventaja de A* es que puede reducir considerablemente el tiempo de búsqueda comparado con BFS, especialmente cuando se utilizan heurísticas informativas como `ignoreDeleteListsHeuristic`.

HTN

- **hierarchicalSearch(problem, hlas):**

En este caso la función implementa una planificación jerárquica HTN usando búsqueda en amplitud sobre refinamientos de planes, este algoritmo comienza con acciones de alto nivel (HLAs) y en cada paso reemplaza la primera acción no primitiva por uno de los refinamientos hasta obtener un plan compuesto únicamente por acciones primitivas que logre el objetivo. Su complejidad temporal en el peor caso es aproximadamente ($O(r^d)$), donde r es el número promedio de refinamientos posibles y d la profundidad jerárquica del plan, ya que debe explorar distintas combinaciones de refinamientos. La complejidad espacial también es ($O(r^d)$) porque almacena múltiples planes parciales en la frontera de búsqueda. El algoritmo es completo siempre que todos los refinamientos posibles sean explorados y exista una descomposición válida del problema pero no siempre es óptimo, ya que la primera descomposición válida encontrada no necesariamente corresponde al plan de menor costo. La principal ventaja del enfoque HTN es que reduce significativamente la complejidad del problema al trabajar con tareas de alto nivel y conocimiento específico del dominio por lo que se puede decir que guía la búsqueda de manera mucho más estructurada que los planificadores clásicos.

- **build_htn_hierarchy(problem):**

Esta función construye la jerarquía de tareas HTN para el dominio de rescate, creando acciones de alto nivel como Navigate, PrepareSupplies, ExtractPatient y FullRescueMission. Para hacerlo genera refinamientos específicos usando información del estado inicial, calcula rutas mediante BFS y conecta las HLAs con acciones primitivas del dominio. Su complejidad temporal es aproximadamente $O(p(V+E+a))$, donde p es el número de pacientes, $(V+E)$ corresponde al costo de las búsquedas BFS sobre el mapa y a al número de acciones instanciadas revisadas para encontrar las acciones compatibles. La complejidad espacial es aproximadamente $O(a+p)$, porque almacena acciones instanciadas, rutas y refinamientos jerárquicos. Esta función no tiene completitud ni optimalidad por sí sola, ya que únicamente construye la jerarquía usada por el planificador HTN. Sin embargo, es muy importante porque organiza el problema en tareas de alto nivel y reduce considerablemente la complejidad de la búsqueda, haciendo que el planeador trabaje de forma más estructurada y cercana a cómo un humano planearía una misión de rescate.

Tabla comparativa

Layout	Algoritmo	Heurística	Estados explorados	Longitud del plan	Tiempo
tinyBase	forwardBFS	—	345	9 acciones	0.018 s
tinyBase	backwardSearch	—	0	9 acciones	0.038 s
tinyBase	A*	ignorePreconditions	252	9 acciones	1.123 s
tinyBase	A*	ignoreDeleteLists	25	9 acciones	3.169 s
openRescue	forwardBFS	—	12982	26 acciones	12.775 s
openRescue	backwardSearch	—	0	26 acciones	60.998 s

En conclusión, Los resultados muestran diferencias entre los algoritmos de planificación. forwardBFS encontró planes óptimos en todos los casos, pero exploró una gran cantidad de estados, especialmente en mapas grandes como openRescue, donde el costo computacional aumentó considerablemente. backwardSearch logró encontrar planes con la misma longitud explorando menos estados, ya que trabaja desde el objetivo y solo considera acciones relevantes; sin embargo, en algunos casos el tiempo de ejecución fue mayor debido al costo de manejar regresiones y objetivos parciales. Por otro lado, A* redujo significativamente la cantidad de estados explorados gracias al uso de heurísticas. Entre las dos heurísticas probadas, ignoreDeleteLists fue la más informativa, ya que solo necesitó explorar 25 estados, mientras que ignorePreconditions exploró 252. Esto demuestra que una heurística más precisa permite guiar mejor la búsqueda y disminuir el espacio explorado, haciendo el algoritmo más eficiente en problemas de planificación complejos.

Punto 6.b:

Durante el desarrollo del taller, utilizamos la IA como herramienta de apoyo para revisar y mejorar algunas partes de la implementación de los algoritmos de planificación. Primero intentamos resolver los problemas por cuenta propia y luego se usó la IA principalmente para corregir errores, entender mejor algunos conceptos y mejorar partes del código. Una de las cosas más útiles fue poder comparar diferentes formas de implementar los algoritmos y entender por qué algunas soluciones eran más eficientes que otras. En general, las correcciones realizadas por la IA fueron más de optimización y organización del código que de construcción completa de la solución. En cada una de las soluciones/optimizaciones que brindó la IA nos tomamos el tiempo de comprender en su totalidad lo que había hecho, por eso en algunos casos fue necesario revisar y adaptar el código para comprender completamente su funcionamiento y asegurar de que se ajustara a lo que pide el taller.